

El problema: cupos bajo presión y sistemas frágiles

Cuando la demanda llega junta, muchas solicitudes compiten por pocos lugares en el mismo instante (*flash crowd*): al abrir inscripciones en SIU Guaraní, al armar un partido o al confirmar un asado.

- Una implementación síncrona lee y escribe **la misma fila**: se agotan las conexiones, crece la latencia y el usuario reintenta.
- Entre leer y escribir se cuelan otras peticiones: **sobreasignación** y reservas duplicadas.
- Cada organizador reimplementa esta lógica sobre su propia base: **si esa base cae, pierde quién está inscripto**, justo en el pico.

Objetivo: un servicio que cualquier sistema consume por HTTP, que no sobreasigna ni duplica, y que es la fuente de verdad que le permite al consumidor seguir operando si falla su propia base.

Tres dominios, un mismo invariante

Inscripción a materias

plataforma educativa

40 cupos

2.000 alumnos a la vez

Partido de fútbol 5

liga barrial

10 cupos

un grupo de 30

Asado del sábado

grupo de amigos

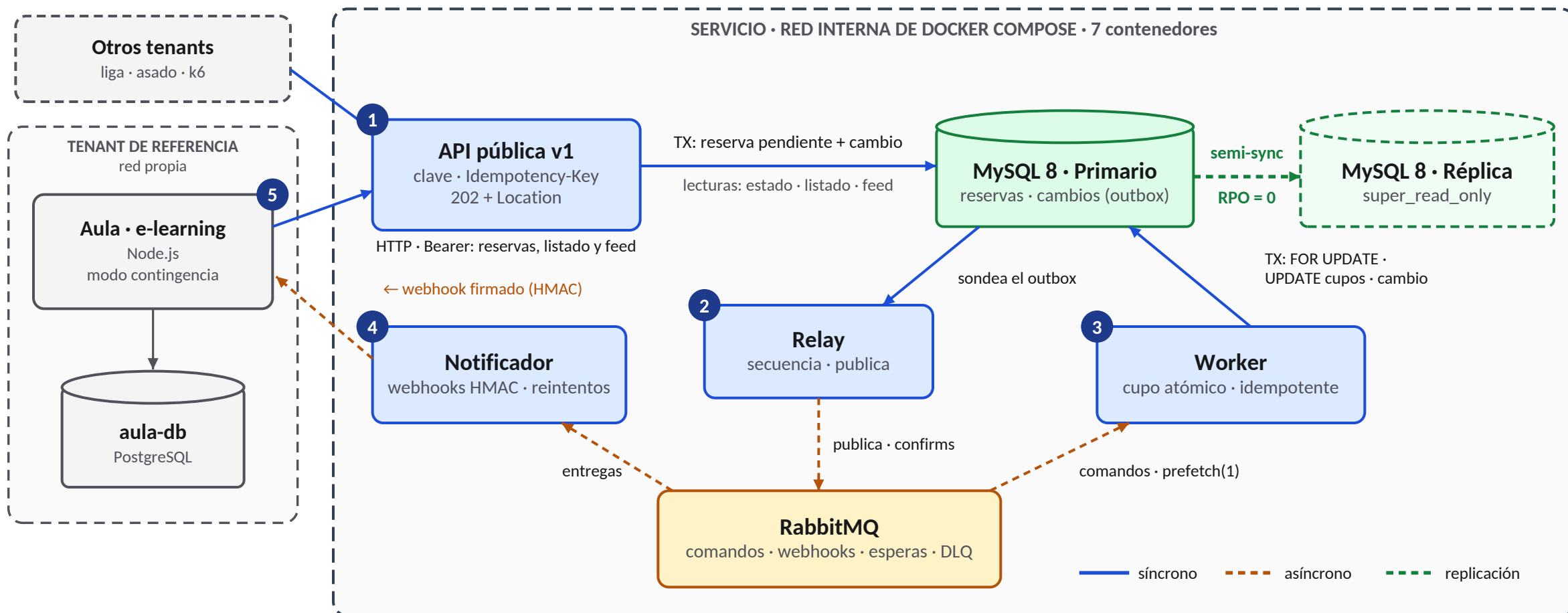
25 lugares

confirmación por lista

Lo que el servicio garantiza en todos

- reservas confirmadas $\leq$ **capacidad**
- a lo sumo **una reserva activa** por participante
- reintentar **nunca duplica** (Idempotency-Key)
- todo lo demás es **metadata opaca** del tenant

La solución: API multi-tenant sobre una cola y un outbox



1 Admisión

Autentica al tenant, guarda la reserva pendiente y su cambio en una transacción y responde 202.

2 Relay

Asigna la secuencia del feed y publica cada cambio con publisher confirms.

3 Worker

Decide de a uno: UPDATE condicional atómico; idempotente ante reintentos.

4 Notificador

Webhook firmado con HMAC; reintentos escalonados; la DLQ no pierde datos.

5 Feed

El tenant lee /v1/cambios desde su punto de control: contingencia y reconciliación.

Continuidad operativa y resiliencia

Si falla la base de datos del tenant

1 • Contingencia

Su aplicación lee inscriptos y cupos desde la API y sigue reservando: la admisión nunca dependió de su base.

2 • Retención

Los webhooks que no puede procesar esperan en colas con TTL; si se agotan, van a la DLQ sin pérdida: el feed conserva todo.

3 • Reconciliación

Al volver, lee /v1/cambios desde su punto de control y aplica cada cambio si su versión es mayor: su base converge.

Si falla un componente del servicio

rabbitmq o relay

La admisión sigue; el outbox retiene cada cambio y el barredor republica lo que se haya perdido.

worker

Los comandos esperan en la cola; la reentrega es idempotente: nunca se descuenta dos veces.

mysql-primary

La API responde 503 + Retry-After; la réplica semi-sincrónica tiene todo lo confirmado (RPO = 0); promoción manual.

api

El tenant reintenta con la misma Idempotency-Key: misma reserva, sin duplicado.

Criterio de aceptación y demo de la defensa: 500 solicitudes contra una materia de 10 cupos → **10 confirmadas · 0 duplicadas · 0 cupos negativos**, con **aula-db** detenida a mitad de la ráfaga, y al terminar la base del tenant idéntica al estado del servicio.